

Análisis de Código Concurrente Generado por VibeCoding

Caso de estudio: Sheriffsss — videojuego 2D en Java

Programación Concurrente — 1° Cuatrimestre 2026

Fecha de entrega: 27 de mayo de 2026

Herramienta: Claude Code (CLI) — Lenguaje: Java 17+

1. Introducción

1.1 Objetivo

Este trabajo evalúa el código de un videojuego 2D concurrente (*Sheriffsss*) generado con la herramienta de *VibeCoding* Claude Code (CLI de Anthropic). El objetivo es contrastar el artefacto producido por el modelo de lenguaje con métricas reconocidas de calidad de software, con especial atención a la corrección de la implementación concurrente.

1.2 ¿Qué es VibeCoding?

VibeCoding es una modalidad de desarrollo en la que el programador describe en lenguaje natural —mediante prompts— lo que quiere construir, y un modelo de lenguaje de gran escala (LLM) genera el código fuente. El desarrollador evalúa el resultado, refina el prompt y repite el ciclo. Permite crear prototipos funcionales rápidamente, pero traslada decisiones de diseño al modelo, lo que puede impactar la calidad estructural y la correctitud concurrente del resultado.

1.3 Marco teórico

Concurrencia en Java

Java provee concurrencia mediante monitores intrínsecos (synchronized), variables volátiles (volatile) y primitivas de alto nivel en `java.util.concurrent`. En aplicaciones gráficas Swing/AWT se suma la regla del Event Dispatch Thread (EDT): toda mutación de la interfaz debe ocurrir en el EDT, y todo trabajo CPU-intensivo debe salir de él para no bloquear la UI.

Métricas de calidad aplicadas

- Complejidad Ciclomática (CCN) [1]: cuantifica los caminos independientes de una función; mayor CCN implica mayor dificultad de testeo y más probable presencia de defectos.
- Maintainability Index (MI) [2]: combina volumen de Halstead, CCN y líneas de código en un escaler 0–100. Por debajo de 10, el código se considera inmantenible.
- Análisis estático de defectos concurrentes con SpotBugs (categoría MT_CORRECTNESS) [3].
- Perfilado en ejecución: CPU, memoria residente (RSS) y dump de threads con `jcmd`, herramienta oficial del JDK [4].
- Detección de God Class según el catálogo de bad smells de Fowler [5] y la heurística de Riel [6].

2. Desarrollo

2.1 Herramienta y modelo utilizados

Item	Valor
Herramienta de VibeCoding	Claude Code (CLI de Anthropic)
Modelos utilizados	Mixto: Claude Sonnet 4.x y Claude Opus 4.x según iteración
Lenguaje objetivo	Java 17+ (compilado con Amazon Corretto 20)
Dependencias externas	Ninguna — solo JDK estándar (Swing/AWT, javax.sound.sampled)

2.2 Prompt inicial y evolución del proyecto

El desarrollo siguió un esquema iterativo típico del VibeCoding: un prompt inicial acotado para obtener un esqueleto mínimo jugable, seguido de sucesivas rondas que incorporaron features y refinaron lo existente.

Prompt inicial: "Quiero hacer un juego 2D top-down sencillo en Java usando solo Swing/AWT, sin librerías externas. La idea es un sheriff en un mapa estilo western. Necesito una ventana JFrame con un JPanel, un game loop en su propio Thread (separado del EDT), un mapa por tiles, un jugador controlado con WASD con cámara que lo siga, todo en un único paquete SheriffsssPackage. Sin enemigos, sin audio, sin menús — esqueleto mínimo que pueda compilar, correr y mover el personaje."

Las iteraciones posteriores se organizaron en bloques temáticos:

Bloque	Foco	Relevancia concurrente
A — Combate y enemigos	Gameplay principal	Baja
B — Mundo y ambientación	Ciclo día/noche, iluminación	Baja
C — Audio	SFX simultáneos	Alta: AudioManager thread-safe (synchronized)
D — Tutorial	Tutorial paso a paso	Alta: TutorialThread + AtomicBoolean
E — Interfaz	Menú, settings, saves	Media: SwingUtilities.invokeLater
F — Debug	Overlays internos	Baja
G — Pulido	Shutdown, portabilidad	Alta: volatile, shutdown hook, Thread.join

Total estimado: **25–40 prompts principales**, más sub-prompts de ajuste que suman aproximadamente **60–100 refinamientos** en total.

2.3 Panorama general del código generado

Métrica	Valor
Archivos .java	46 (en paquete único SheriffsssPackage)

Métrica	Valor
NLOC efectivos (Lizard)	6 536
Funciones totales	913
Dependencias externas	Ninguna

2.4 Arquitectura concurrente

El análisis del fuente identificó los siguientes hilos gestionados explícitamente por el código:

Hilo	Origen	Mecanismo de sincronización
SheriffsssGameLoop	Game.java — new Thread(this, ...)	volatile gameThread y shuttingDown
TutorialThread	Extiende Thread; lanzado por TrainingMode	AtomicBoolean skipRequested/finished; sleep con interrupción cooperativa
SheriffsssGameShutdown	Main.java — Runtime.addShutdownHook	Llama a Game.shutdown() que es synchronized

Adicionalmente, el código usa el EDT de Swing vía `SwingUtilities.invokeLater` y los hilos `Direct Clip` que `javax.sound.sampled` crea por cada clip activo en reproducción simultánea.

2.5 Análisis crítico

Fortalezas

- Game loop aislado del EDT y marcado con `volatile`; cumple la regla canónica de Swing de separar simulación de interfaz.
- Shutdown ordenado: shutdown hook + `Thread.join` con timeout evita el leak de threads daemon al cerrar la ventana.
- `AudioManager` envuelve todas las mutaciones en `synchronized this` — monitor único por agregado mutable; no se detectan secciones críticas faltantes.
- Uso correcto de `AtomicBoolean` en `TutorialThread` para señalización sin busy-waiting protegido por locks.

Debilidades

- **God Class:** `Game.java` concentra 1 261 NLOC y 170 funciones. Deteriora el MI a 4.3 (inmantenible) y configura el bad smell "Large Class" de Fowler [5, 6]. Patrón frecuente en código LLM cuando el prompt no especifica arquitectura.
- **Acumulador float en game loop:** SpotBugs detectó `FL_FLOATS_AS_LOOP_COUNTERS` en `Game.run` — usar `double` como acumulador de tiempo puede generar drift de precisión en sesiones largas.
- **Sin separación render/lógica:** simulación y render corren acoplados en el mismo bucle, limitando la escalabilidad.
- **Bug de portabilidad (macOS):** hotspot del cursor declarado en (15,15) para imagen 15×15; LWAWT de Apple lo rechaza con `IndexOutOfBoundsException`. El modelo no detectó esto porque la generación asumió Windows.

3. Resultados

3.1 Complejidad Ciclomática (CCN)

Herramienta: **Lizard 1.22.1** (CLI, MIT). Implementación de referencia de McCabe [1]; no requiere instrumentación del binario; soporta Java sin configuración.

Métrica global	Valor
Funciones analizadas	913
CCN promedio	2.3 — Excelente
Funciones con CCN > 10 (umbral McCabe)	20 (2.2%)
Funciones con CCN > 15 (warning Lizard)	6 (0.66%)

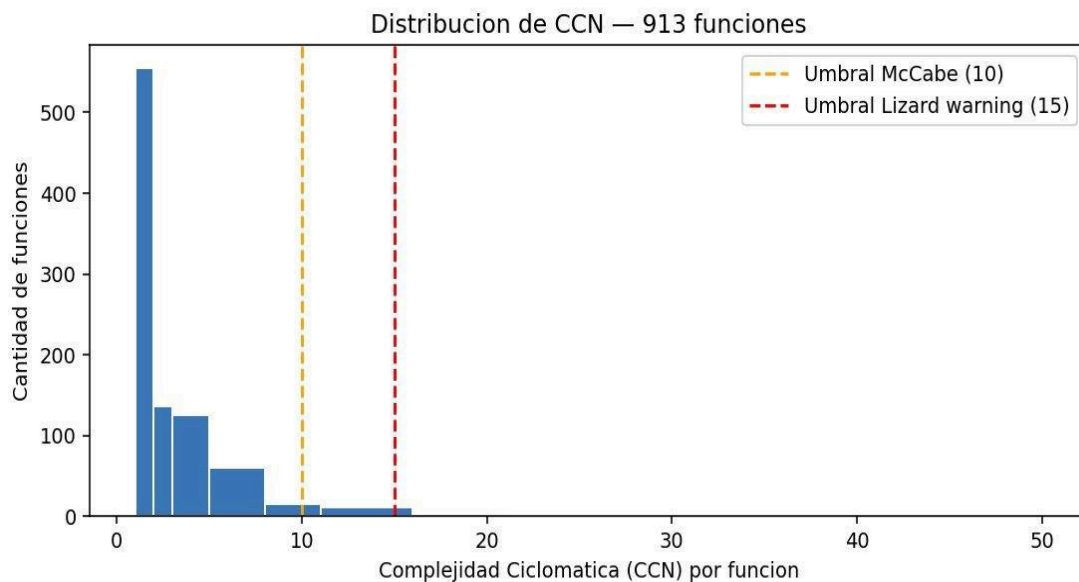


Figura 1. Distribución de complejidad ciclomática — 913 funciones

Las 6 funciones con CCN crítico:

Función	CCN	Archivo
GameInput::keyPressed	44 (high risk)	GameInput.java:186
GameInput::keyReleased	21	GameInput.java:296
Game::updateCursor	21	Game.java:1296
Game::updateSettings	18	Game.java:1026
GameRenderer::drawDebugWorld	16	GameRenderer.java:563
Game::attemptFireProjectileWeapon	16	Game.java:698

La complejidad promedio por función es muy buena. Las concentraciones críticas aparecen en dispatchers de input y settings, que son estructuralmente complejos; pero keyPressed con CCN=44 supera el umbral high risk (>30) de McCabe y amerita refactor a tabla de despacho.

3.2 Mantenibilidad (MI)

Herramienta: script propio *compute_mi.py* sobre el CSV de Lizard, aplicando la fórmula de Microsoft, variante normalizada de Oman & Hagemeister [2].

Banda	Archivos	Detalle
Inmantenible (MI < 10)	2	Game.java (4.3), GameRenderer.java (4.8)
Moderado (10–19)	1	TrainingMode.java (10.0)
Bueno (>= 20)	38	Resto del proyecto
Promedio global	—	41.7

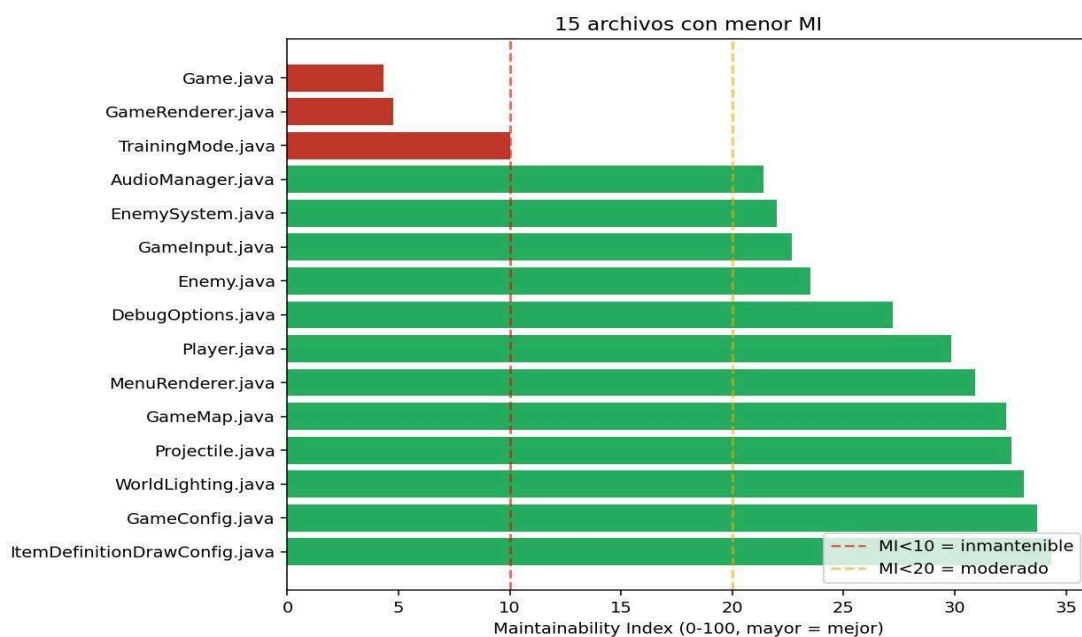


Figura 2. Maintainability Index — 15 archivos más críticos

El problema no es la complejidad por función sino la acumulación de funciones por archivo: Game.java (170 fns), GameRenderer.java (91), TrainingMode.java (89). El MI penaliza fuertemente el ln(LOC) del archivo. Es un síntoma habitual del código LLM que tiende a entregar artefactos monolíticos salvo que el prompt restrinja la arquitectura.

3.3 Rendimiento — Composición de threads

Herramienta: **jcmd Thread.print** (JDK oficial [4]). Tres snapshots a t=5 s (menú), t=10 s y t=14 s desde el arranque en modo JIT con Amazon Corretto 20.

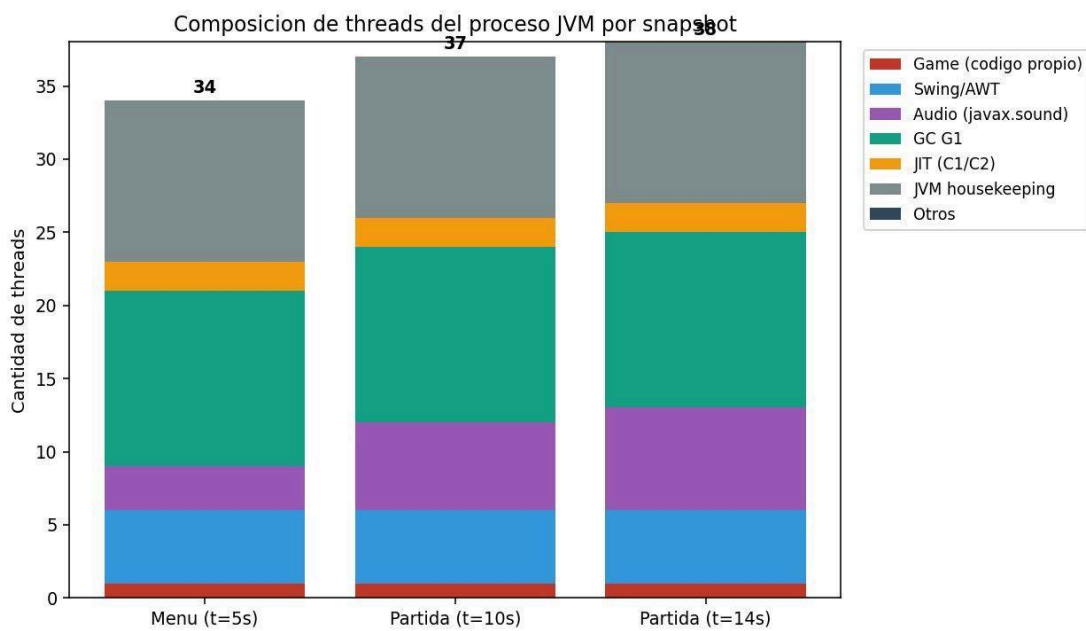


Figura 3. Composición de threads del proceso JVM por snapshot

Categoría	t=5s (menú)	t=10s (partida)	t=14s (partida)
Game (código propio)	1	1	1
Swing/AWT	4	4	4
Audio (javax.sound)	3	6	7
GC G1	12	12	12
JIT (C1+C2)	2	2	2
JVM housekeeping	11	11	11
Otros	1	1	1
Total	34	37	38

El código del juego mantiene un único thread propio activo en estado estable (SheriffsssGameLoop). El único factor de crecimiento durante la partida son los Direct Clip de audio (uno por sample simultáneo). Esto explica la ausencia de bugs de concurrencia: no existe paralelismo real en la simulación, solo coordinación entre game loop, EDT y audio.

3.4 Rendimiento — CPU y Memoria

Herramientas: **ps (CPU%/RSS) + jcmd GC.heap_info**. Bajo overhead, disponibles nativamente, suficientes para snapshots discretos. CPU% puede superar 100% porque POSIX suma el uso de cada hilo en cada core físico.

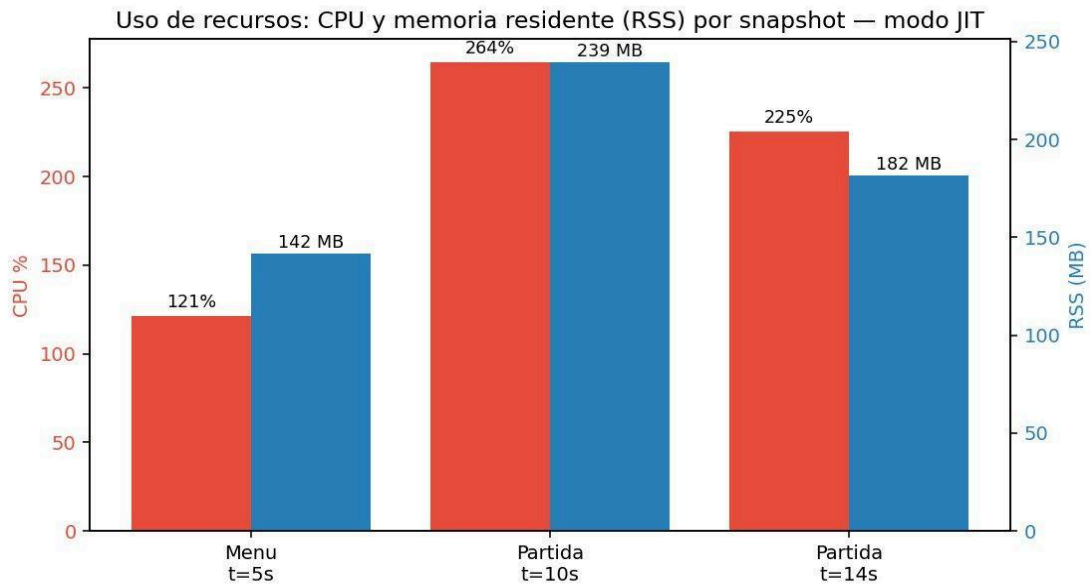


Figura 4. Uso de CPU y memoria residente (RSS) — modo JIT

Snapshot	CPU %	RSS	Heap G1
t=5s (menú)	121%	142 MB	~56 MB
t=10s (partida)	264%	245 MB	~88 MB
t=14s (partida)	225%	182 MB	~65 MB (tras GC)

- CPU > 100%: GC paralelo G1, compiladores JIT y audio usan múltiples cores aunque el game loop sea single-threaded.
- Pico RSS (245 MB) corresponde a la carga de sprites y clips de audio en memoria (preloading agresivo del AssetManager).
- Descenso de RSS y heap entre t=10s y t=14s: G1 recupera memoria efectivamente; sin indicios de memory leak en la ventana observada.

3.5 Correctitud Concurrente

Herramienta: **SpotBugs 4.8.6** en modo `-textui -effort:max` sobre los `.class` compilados.
 Categoría clave: `MT_CORRECTNESS` (detectores de double-checked locking, sincronización inconsistente, locks no liberados, etc.).

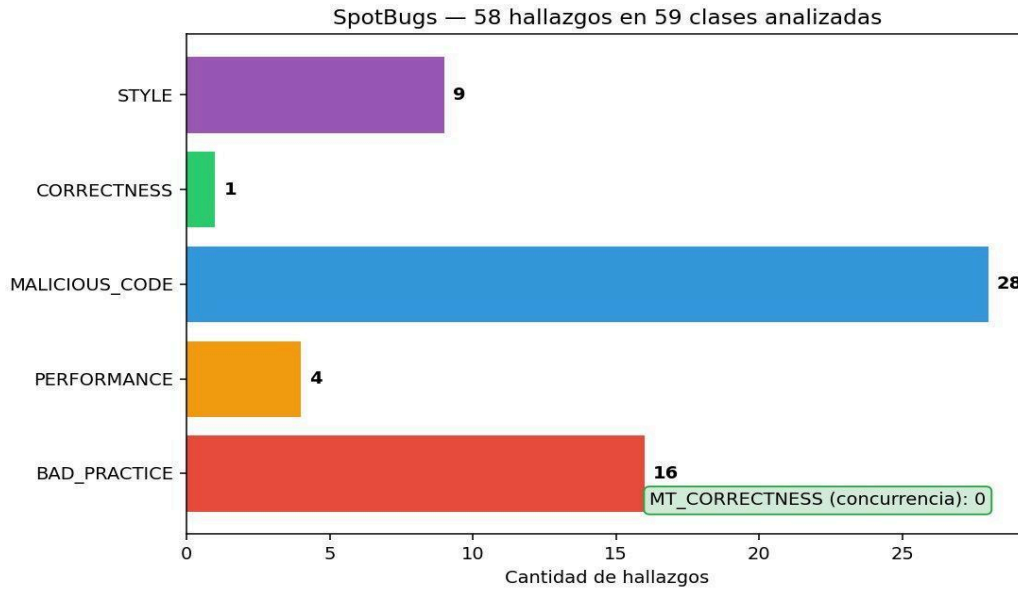


Figura 5. Hallazgos SpotBugs por categoría — 59 clases analizadas

Categoría SpotBugs	Hallazgos
MALICIOUS_CODE	28
BAD_PRACTICE	16
STYLE	9
PERFORMANCE	4
CORRECTNESS	1
MT_CORRECTNESS (concurrency)	0
Total	58

MT_CORRECTNESS = 0: no se detectaron defectos de concurrencia clásicos. Todo estado mutable compartido entre threads es accedido con synchronized, volatile o variable atómica.

Único CORRECTNESS: FL_FLOATS_AS_LOOP_COUNTERS en Game.run — acumulador de tiempo usa double, puede producir drift en sesiones largas. Recomendación: migrar a long con System.nanoTime().

Nota sobre MALICIOUS_CODE: SpotBugs usa esta categoría para patrones de encapsulación débil, no código malicioso. De los 28 hallazgos, los más relevantes para concurrencia son: (a) TutorialThread recibe su lista de pasos por referencia sin copiarla — seguro hoy, frágil ante refactor; (b) campos estáticos públicos en GameConfig (SCREEN_WIDTH, etc.) mutables sin sincronización — canal de visibility race no tipificado en MT_CORRECTNESS.

3.6 Tokens y Refinamientos

Métrica	Valor	Método
Bytes de código fuente	305 170	Medición directa
Tokens (código final)	~76 300	Proxy bytes/4 (regla de oro BPE)
Tokens totales estimados	250 000 – 750 000	Rango 3x–10x sobre el artefacto final
Prompts principales	25 – 40	Reconstrucción por bloques temáticos
Total con sub-prompts	~60 – 100	Estimación con intervalo explícito

La trazabilidad exacta de tokens no se preservó durante el desarrollo. El proxy bytes/4 es aplicable a la familia Claude porque el tokenizador BPE produce una densidad comparable para texto en inglés y código Java. El intervalo de refinamientos se reporta como rango para no falsificar una precisión que no se tiene.

4. Conclusiones

4.1 Síntesis cuantitativa

Métrica	Resultado	Evaluación
CCN promedio	2.3	Excelente
CCN máximo	44 (GameInput.keyPressed)	Crítico — refactor recomendado
MI promedio	41.7	Bueno
MI mínimo	4.3 (Game.java)	Inmantenible — God Class
Hilos propios (estable)	1 game loop + 1 tutorial efímero	Correcto para la escala
Pico CPU	264%	Aceptable
Pico RSS	245 MB	Aceptable
MT_CORRECTNESS (SpotBugs)	0 hallazgos	Excelente
Refinamientos estimados	60 – 100	—

4.2 Correctitud concurrente

La implementación es conservadora pero correcta. El modelo de un único game-loop thread evita la mayor parte de la complejidad concurrente, y las zonas verdaderamente multi-thread (audio, shutdown, tutorial) están sincronizadas con la primitiva adecuada: `synchronized` para invariantes compuestas, `volatile` para flags de visibilidad, `AtomicBoolean` para señalización sin sección crítica.

Esto matiza la intuición de que los LLM producen código concurrente defectuoso: cuando el problema admite una solución conservadora, el modelo la implementa correctamente. El riesgo real aparece en la encapsulación (referencias expuestas, campos estáticos mutables) más que en los primitivos de sincronización.

4.3 Calidad estructural y God Class

La debilidad principal no está en la concurrencia sino en la **arquitectura**: `Game.java` (MI=4.3) y `GameRenderer.java` (MI=4.8) en zona inmantenible por acumulación monolítica. Riel [6] y Fowler [5] son contundentes: la God Class viola el principio de responsabilidad única, dificulta los tests unitarios e incrementa el acoplamiento. El código generado por LLM tiende a maximizar funcionalidad por respuesta a costa de modularidad, salvo que el prompt restrinja explícitamente la arquitectura.

4.4 Ventajas y limitaciones del VibeCoding

Ventajas

- Velocidad de entrega de un prototipo funcional con sincronización correcta, sin expertise profundo en el Java Memory Model.
- Selección razonable de primitivas: `volatile` para flags, `AtomicBoolean` para señalización, `synchronized` para invariantes compuestas.

- Patrón de shutdown ordenado (hook + join con timeout) aplicado por defecto.

Limitaciones

- Sesgo monolítico: tendencia a God Class si el prompt no especifica arquitectura de paquetes.
- Sesgo de plataforma: defectos silenciosos al portar entre sistemas operativos (bug del cursor en macOS).
- Subutilización del paralelismo: el modelo eligió la solución conservadora sin explorar paralelización de IA de enemigos o render asíncrono.
- Falta de trazabilidad nativa de tokens y refinamientos en el flujo de Claude Code.

4.5 Mejoras propuestas para una entrega posterior

Las siguientes mejoras están ordenadas por impacto y costo de implementación:

Prioridad	Mejora	Justificación
Alta	Refactor de Game.java (Extract Class)	Descomponer en GameStateManager, WindowManager, GameLoop, WeaponController y CursorController. Llevaría el MI de 4.3 a >20 y reduciría el CCN global.
Alta	Migrar acumulador de tiempo a long + System.nanoTime()	Elimina FL_FLOATS_AS_LOOP_COUNTERS y el drift de precisión en sesiones largas.
Media	Copias defensivas en TutorialThread y GameConfig	Elimina los riesgos latentes de concurrencia: lista de pasos aliasada y campos estáticos mutables sin sincronización.
Media	Refactor de keyPressed a tabla de despacho	Reduce CCN de 44 a <10 y facilita agregar bindings nuevos sin escalar la complejidad.
Baja	Instrumentar el workflow para trazabilidad de tokens	Habilita reportabilidad precisa de tokens y refinamientos en futuras entregas académicas.
Baja	Separar render de lógica en el game loop	Permite frames variables e independientes de la velocidad de simulación, mejorando la escalabilidad.

Un refactor asistido con la misma herramienta de VibeCoding, pero con un prompt que especifique explícitamente la arquitectura de paquetes, podría reducir Game.java a ~400 NLOC y elevar su MI por encima de 20 en pocas iteraciones.

5. Referencias

[1] T. J. McCabe, "A complexity measure", IEEE Transactions on Software Engineering, vol. SE-2, n.o 4, pp. 308–320, dic. 1976.

[2] P. Oman y J. Hagemester, "Metrics for assessing a software system's maintainability", en Proc. Conference on Software Maintenance, IEEE, 1992, pp. 337–344.

[3] D. Hovemeyer y W. Pugh, "Finding bugs is easy", ACM SIGPLAN Notices, vol. 39, n.o 12, pp. 92–106, 2004.

[4] Oracle Corporation, JDK 17 Tools Specification — jcmd, 2021. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/specs/man/jcmd.html>

[5] M. Fowler, Refactoring: Improving the Design of Existing Code, 2.a ed. Boston, MA: Addison-Wesley, 2018.

[6] A. J. Riel, Object-Oriented Design Heuristics. Reading, MA: Addison-Wesley, 1996.